Rajalakshmi Engineering College

Name: Sanjay Ram K
Email: 240801296@rajalakshmi.edu.in
Roll no: 240801296
Phone: 8610528612
Branch: REC
Department: I ECE AF
Batch: 2028
Degree: B.E - ECE

Scan to verify results

NeoColab_REC_CS23231_DATA STRUCTURES

REC_DS using C_Week 2_PAH

Attempt : 1
Total Mark : 50
Marks Obtained : 50

Section 1 : Coding

1. Problem Statement

Riya is developing a contact management system where recently added contacts should appear first. She decides to use a doubly linked list to store contact IDs in the order they are added. Initially, new contacts are inserted at the front of the list. However, sometimes she needs to insert a new contact at a specific position in the list based on priority.

Help Riya implement this system by performing the following operations:

Insert contact IDs at the front of the list as they are added.Insert a new contact at a given position in the list.

Input Format

The first line of input consists of an integer N, representing the initial size of the linked list.

The second line consists of N space-separated integers, representing the values of the linked list to be inserted at the front.

The third line consists of an integer position, representing the position at which the new value should be inserted (position starts from 1).

The fourth line consists of integer data, representing the new value to be inserted.

Output Format

The first line of output prints the original list after inserting initial elements to the front.

The second line prints the updated linked list after inserting the element at the specified position.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 4
10 20 30 40
3
25
Output: 40 30 20 10
40 30 25 20 10

Answer

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
```

```
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

void insertFront(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        newNode->next = *head;
        (*head)->prev = newNode;
        *head = newNode;
    }
}


void insertAtPosition(struct Node** head, int position, int data) {
    struct Node* newNode = createNode(data);
    if (position == 1) {
        insertFront(head, data);
        return;
    }

    struct Node* current = *head;
    for (int i = 1; i < position - 1 && current != NULL; i++) {
        current = current->next;
    }

    if (current == NULL) {
        printf("Position out of bounds\n");
        return;
    }

    newNode->next = current->next;
    newNode->prev = current;
    if (current->next != NULL) {
        current->next->prev = newNode;
    }
```

```
    current->next = newNode;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int N, position, data;


    scanf("%d", &N);
    struct Node* head = NULL;

    for (int i = 0; i < N; i++) {
        scanf("%d", &data);
        insertFront(&head, data);
    }


    printList(head);


    scanf("%d %d", &position, &data);


    insertAtPosition(&head, position, data);


    printList(head);

    return 0;
}
```

Status : Correct ***Marks : 10/10***

2. Problem Statement

Rohan is a software developer who is working on an application that processes data stored in a Doubly Linked List. He needs to implement a feature that finds and prints the middle element(s) of the list. If the list contains an odd number of elements, the middle element should be printed. If the list contains an even number of elements, the two middle elements should be printed.

Help Rohan by writing a program that reads a list of numbers, prints the list, and then prints the middle element(s) based on the number of elements in the list.

Input Format

The first line of the input consists of an integer n the number of elements in the doubly linked list.

The second line consists of n space-separated integers representing the elements of the list.

Output Format

The first line prints the elements of the list separated by space. (There is an extra space at the end of this line.)

The second line prints the middle element(s) based on the number of elements.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
20 52 40 16 18

Output: 20 52 40 16 18
40

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
struct Node {
    int data;
    struct Node* next;
    struct Node* prev;
};


struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    newNode->prev = NULL;
    return newNode;
}


void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}


void findMiddle(struct Node* head, int n) {
    struct Node* slow = head;
    struct Node* fast = head;


    if (n % 2 == 1) {
        while (fast != NULL && fast->next != NULL) {
            slow = slow->next;
            fast = fast->next->next;
        }
        printf("%d\n", slow->data);
    } else {

        int count = 0;
```

```
        while (fast != NULL && fast->next != NULL) {
            slow = slow->next;
            fast = fast->next->next;
            count++;
        }
        printf("%d %d\n", slow->prev->data, slow->data);
    }
}


int main() {
    int n;
    scanf("%d", &n);

    int data;
    struct Node* head = NULL;
    struct Node* tail = NULL;


    for (int i = 0; i < n; i++) {
        scanf("%d", &data);
        struct Node* newNode = createNode(data);
        if (head == NULL) {
            head = newNode;
            tail = newNode;
        } else {
            tail->next = newNode;
            newNode->prev = tail;
            tail = newNode;
        }
    }


    printList(head);


    findMiddle(head, n);

    return 0;
}
```

Status : Correct *Marks : 10/10*

3. Problem Statement

Pranav wants to clockwise rotate a doubly linked list by a specified number of positions. He needs your help to implement a program to achieve this. Given a doubly linked list and an integer representing the number of positions to rotate, write a program to rotate the list clockwise.

Input Format

The first line of input consists of an integer n, representing the number of elements in the linked list.

The second line consists of n space-separated linked list elements.

The third line consists of an integer k, representing the number of places to rotate the list.

Output Format

The output displays the elements of the doubly linked list after rotating it by k positions.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 4 5
1

Output: 5 1 2 3 4

Answer

```
#include <stdio.h>
#include <stdlib.h>


struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};
```

```
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = newNode->next = NULL;
    return newNode;
}


void appendNode(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
    newNode->prev = temp;
}


void rotateClockwise(struct Node** head, int k) {
    if (*head == NULL || k == 0) return;


    struct Node* temp = *head;
    int n = 1;
    while (temp->next != NULL) {
        temp = temp->next;
        n++;
    }


    k = k % n;
    if (k == 0) return;


    temp = *head;
```

```
    for (int i = 1; i < n - k; i++) {
        temp = temp->next;
    }

    struct Node* newHead = temp->next;
    temp->next = NULL;
    newHead->prev = NULL;

    struct Node* oldTail = newHead;
    while (oldTail->next != NULL) {
        oldTail = oldTail->next;
    }
    oldTail->next = *head;
    (*head)->prev = oldTail;

    *head = newHead;
}

void printList(struct Node* head) {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d", temp->data);
        if (temp->next != NULL) {
            printf(" ");
        }
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    int n, k;
    scanf("%d", &n);

    struct Node* head = NULL;
    for (int i = 0; i < n; i++) {
        int data;
        scanf("%d", &data);
        appendNode(&head, data);
```

```
    }

    scanf("%d", &k);

    rotateClockwise(&head, k);
    printList(head);

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Bala is a student learning about the doubly linked list and its functionalities. He came across a problem where he wanted to create a doubly linked list by appending elements to the front of the list.

After populating the list, he wanted to delete the node at the given position from the beginning. Write a suitable code to help Bala.

Input Format

The first line contains an integer N, the number of elements in the doubly linked list.

The second line contains N integers separated by a space, the data values of the nodes in the doubly linked list.

The third line contains an integer X, the position of the node to be deleted from the doubly linked list.

Output Format

The first line of output displays the original elements of the doubly linked list, separated by a space.

The second line prints the updated list after deleting the node at the given position X from the beginning.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
10 20 30 40 50
2
Output: 50 40 30 20 10
50 30 20 10

Answer

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

void pushFront(struct Node** headRef, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *headRef;
    if (*headRef != NULL)
        (*headRef)->prev = newNode;
    *headRef = newNode;
}

void deleteAtPosition(struct Node** headRef, int position) {
    if (*headRef == NULL || position <= 0)
        return;

    struct Node* temp = *headRef;
```

```
    for (int i = 1; temp != NULL && i < position; i++) {
        temp = temp->next;
    }

    if (temp == NULL)
        return;

    if (*headRef == temp)
        *headRef = temp->next;

    if (temp->next != NULL)
        temp->next->prev = temp->prev;

    if (temp->prev != NULL)
        temp->prev->next = temp->next;

    free(temp);
}

void printList(struct Node* head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
}

int main() {
    int N, X;
    scanf("%d", &N);

    int arr[N];
    for (int i = 0; i < N; i++)
        scanf("%d", &arr[i]);

    scanf("%d", &X);

    struct Node* head = NULL;

    for (int i = 0; i < N; i++) {
        pushFront(&head, arr[i]);
    }
```

```
    printList(head);
    printf("\n");

    deleteAtPosition(&head, X);

    printList(head);
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

5. Problem Statement

Tom is a software developer working on a project where he has to check if a doubly linked list is a palindrome. He needs to write a program to solve this problem. Write a program to help Tom check if a given doubly linked list is a palindrome or not.

Input Format

The first line consists of an integer N, representing the number of elements in the linked list.

The second line consists of N space-separated integers representing the linked list elements.

Output Format

The first line displays the space-separated integers, representing the doubly linked list.

The second line displays one of the following:

1. If the doubly linked list is a palindrome, print "The doubly linked list is a palindrome".
2. If the doubly linked list is not a palindrome, print "The doubly linked list is not a palindrome".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5
1 2 3 2 1

Output: 1 2 3 2 1
The doubly linked list is a palindrome

Answer

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* prev;
    struct Node* next;
} Node;

Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->prev = NULL;
    newNode->next = NULL;
    return newNode;
}

void append(Node** head, int data) {
    Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
    Node* temp = *head;
    while (temp->next != NULL)
        temp = temp->next;
    temp->next = newNode;
    newNode->prev = temp;
}

int isPalindrome(Node* head) {
    Node* tail = head;
```

```
    while (tail->next != NULL)
        tail = tail->next;
    while (head != NULL && tail != NULL && head != tail && tail->next != head) {
        if (head->data != tail->data)
            return 0;
        head = head->next;
        tail = tail->prev;
    }
    return 1;
}

int main() {
    int n, val;
    scanf("%d", &n);
    Node* head = NULL;
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        append(&head, val);
    }

    Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }

    if (isPalindrome(head))
        printf(" The doubly linked list is a palindrome\n");
    else
        printf(" The doubly linked list is not a palindrome\n");

    return 0;
}
```

Status : Correct ***Marks : 10/10***